



01211373 Machine Learning and Programming for Industry

# Model Evaluation & Hyperparameter Tuning

Lecture 6 — Classical Machine Learning

---

# Last Week → This Week

## WEEK 5 RECAP

- PCA compresses redundant features into a few components
- k-means forces every point into a cluster
- DBSCAN's noise label doubles as an anomaly flag
- Five weeks of models — but how much do we trust the scores?

## TODAY

- Why a single train/test split can mislead you
- k-fold cross-validation, learning & validation curves
- Grid search: tuning hyperparameters without cheating
- Beyond accuracy: precision, recall, ROC & AUC

# Why This Week Matters



Every model you've built so far reported a score from one train/test split. In safety-relevant automation, that's not enough evidence.

## The scenario to keep in mind all lecture

Your fault detector will decide when to stop a production line. Stopping unnecessarily costs money every time (false alarm); failing to stop when a real fault is present can wreck a machine — or hurt someone (missed detection). “95% accurate on one split” answers neither question. Today is about producing numbers you can defend.

# The Problem with One Train/Test Split



Change nothing but the random split, and your “model quality” number moves. Which one goes in your report?

`random_state=0`

**94.4%**

`random_state=1`

**91.7%**

`random_state=2`

**96.1%**

`random_state=3`

**92.8%**

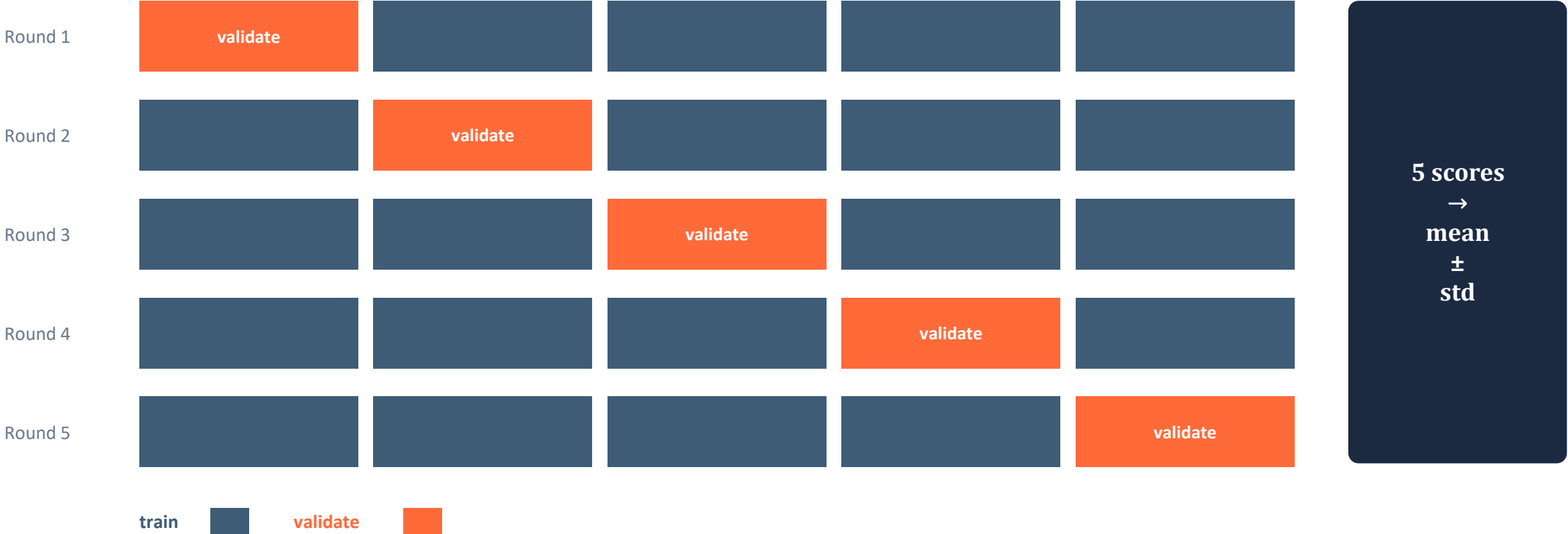
*Same data, same model, same preprocessing — only the split changed. (Illustrative numbers; you'll reproduce this effect in the lab.)*

**The fix:** don't report one number from one split — report the mean and spread across many splits.

# k-Fold Cross-Validation



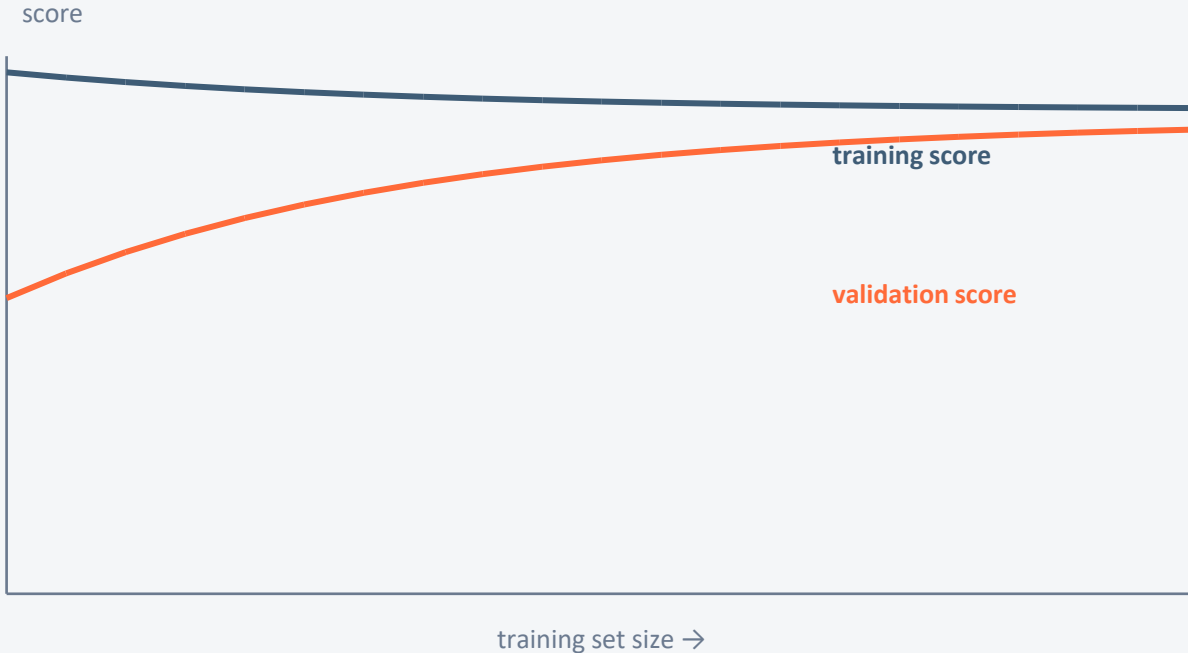
Split the data into k folds. Train on k-1 of them, validate on the held-out one, rotate, and average the k scores.



# Learning Curves: Would More Data Help?



Plot training and validation score against training-set size. The shape tells you whether to collect more data — before you spend weeks doing it.



**Curves converging, both high:** you have enough data.

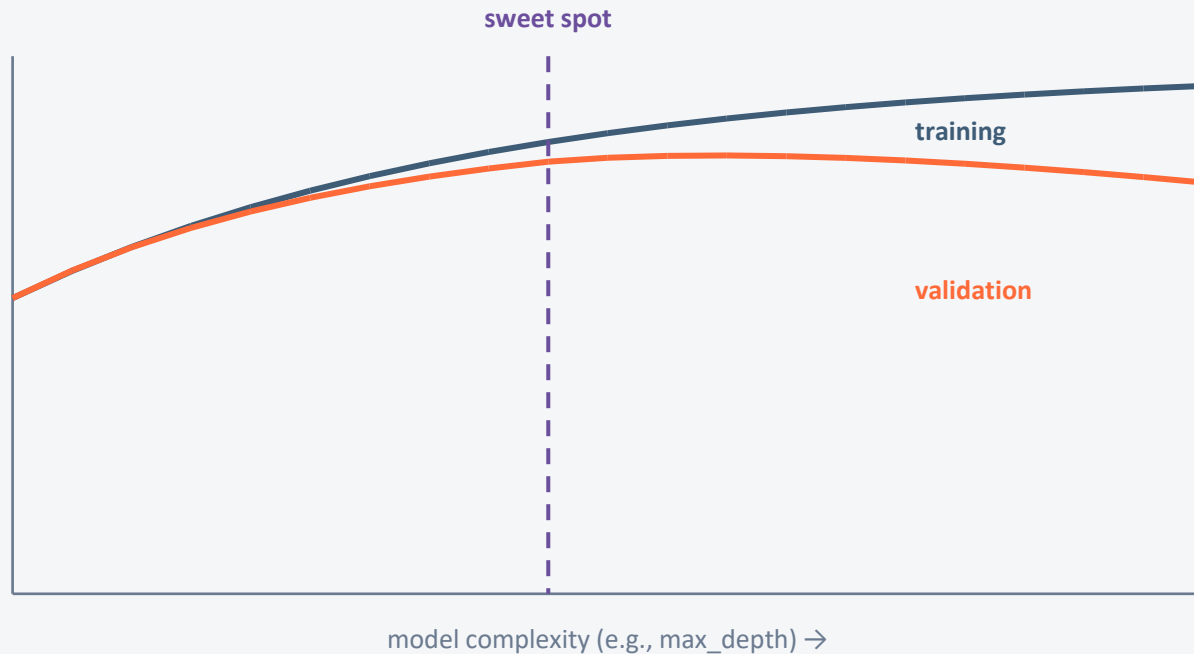
**Large persistent gap:** overfitting — more data (or a simpler model) will help.

**Both low & flat:** underfitting — more data won't save you; the model is too simple.

# Validation Curves: Sweeping One Hyperparameter



Same idea, different x-axis: sweep a hyperparameter (tree depth,  $k$ ,  $\alpha$ ,  $C$ ...) and watch train vs. validation score — Week 3's degree sweep was exactly this.



**Left of the sweet spot:** underfitting — both scores rise together.

**Right of it:** overfitting — training keeps climbing while validation falls away.

# Grid Search: Tuning Without Cheating



GridSearchCV tries every combination of the hyperparameters you list, scoring each with cross-validation — the test set stays untouched until the very end.

```
param_grid = {"clf__max_depth": [3, 5, 8], "clf__n_estimators": [100, 200]}

search = GridSearchCV(pipe, param_grid, cv=5, scoring="f1_macro")

search.fit(X_train, y_train)      # CV happens INSIDE the training set

search.best_params_, search.best_score_
```

**Grids explode fast:**  $3 \times 2$  values  $\times$  5 folds = 30 fits above. For big grids, RandomizedSearchCV samples combinations instead of trying them all.

# Beyond Accuracy: The Confusion Matrix



Accuracy compresses four very different outcomes into one number. For a fault detector, the four cells matter individually.

|                | predicted: normal                                         | predicted: fault                                                |
|----------------|-----------------------------------------------------------|-----------------------------------------------------------------|
| actual: normal | <b>True Negative</b><br>normal, called normal             | <b>False Positive</b><br>false alarm — line stopped for nothing |
| actual: fault  | <b>False Negative</b><br>missed fault — the dangerous one | <b>True Positive</b><br>fault, caught                           |

**Precision:** of the faults you called, how many were real? (controls false alarms)

**Recall:** of the real faults, how many did you catch? (controls missed detections)

# Precision vs. Recall: An Engineering Decision



You can trade one for the other by moving the decision threshold. Where to sit on that trade-off is not a math question — it's an engineering and safety question.

## Favor Precision

Stopping the line is very expensive, faults are recoverable, and inspection is quick. Tolerate a few missed detections to avoid constant false alarms.

## Favor Recall

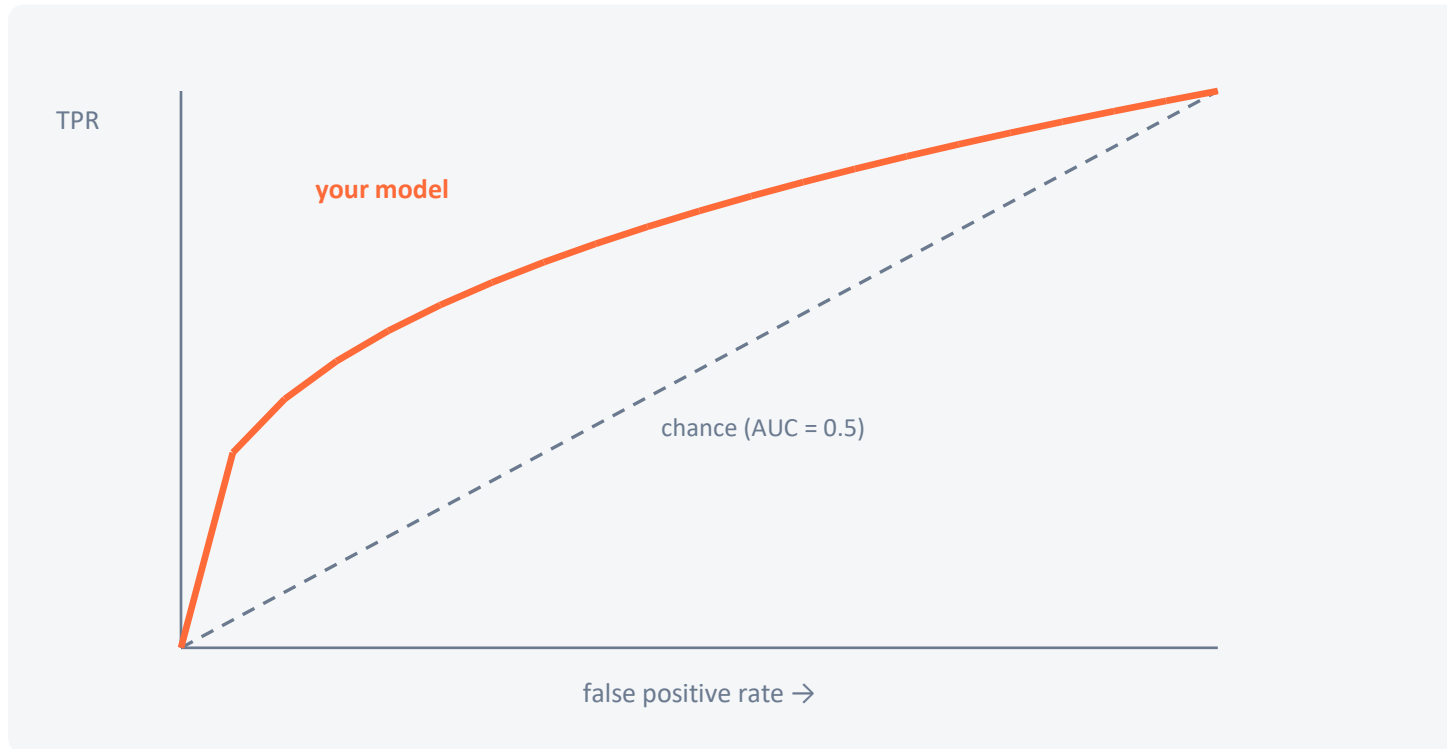
A missed fault damages equipment or endangers people. Accept more false alarms — an unnecessary stop is annoying; a wrecked spindle is not.

*F1 balances the two — but for real deployments, state the trade-off explicitly instead of hiding behind one blended number.*

# ROC Curves & AUC



Sweep the decision threshold from strict to lenient and plot true-positive rate vs. false-positive rate at every point — one curve summarizes every possible threshold.



**AUC** = area under the curve.

1.0 = perfect ranking of faults above normals.

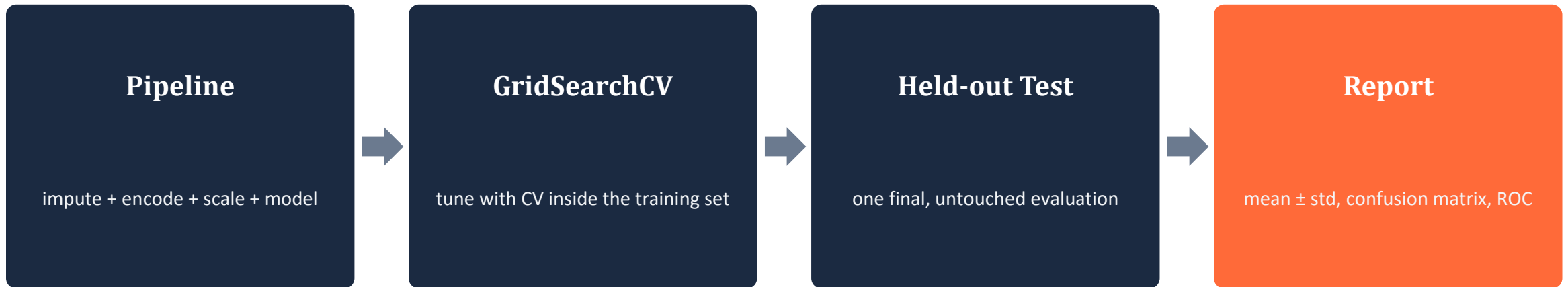
0.5 = coin flip.

Threshold-independent — great for comparing models before you commit to an operating point.

# The Canonical Pattern: Pipeline + CV + Grid Search



Everything from Weeks 2–6 composes into one workflow — and this is exactly what Checkpoint 1 asks you to deliver.



**The one rule that makes it all legitimate:** the test set is touched exactly once, at the very end, after every decision is already made.

# Today's Lab: Evaluate Like You Mean It

1

**Reproduce the one-split problem**

Same model, different random splits, different scores.

2

**Cross-validate the whole pipeline**

Report mean  $\pm$  std instead of one number.

3

**Learning & validation curves**

Diagnose whether more data or different complexity helps.

4

**Grid search, then final test + ROC**

Tune honestly, then spend your one look at the test set.

# Key Takeaways



One split gives you a number; cross-validation gives you evidence.



Tune hyperparameters with CV inside the training set — test data is touched once.



Precision vs. recall is a safety decision. Make it explicitly, not by accident.

NEXT WEEK

## From Neural Network Foundations to PyTorch

Part 2 of the course begins — tensors, autograd, and your first neural network.